

INDEX

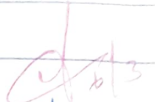
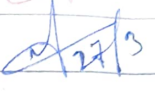
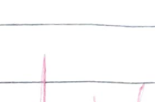
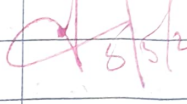
Name : P. Dheeraj Reddy

Subject : OS (Lab)

Std :

Sec. :

Roll No. :

Sl No.	Date	Title	Marks	Teacher's Sign/Remarks
28/03	28/03	Basic C Programs	10	
06/03	06/03	FCFS & SJF		
29/03	29/03	Round robin		
27/03	27/03	Multilevel queue		
10/04	10/04	RMS, EDF, Proportional schedule		
20/04	20/04	Bounded Buffer & Dining Philosophers Problem		
8/5	8/5	Bankers algorithm & Deadlock detection		
15/5/26	15/5/26	Memory Allocation	10	
22/5/26	22/5/26	Page faults		

Q.1) WAP in C to find second smallest element in an array

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("enter number of elements in an array");
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[50];
```

```
    for (int i=0; i<n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int s1 = 1000;
```

```
    int s2 = 1000;
```

```
    for (int j=0; j<n; j++) {
```

```
        if (arr[j] < s1) {
```

```
            s2 = s1;
```

```
            s1 = arr[j];
```

```
        }
```

```
        else if (s2 > arr[j] && s1 != arr[j]) {
```

```
            s2 = arr[j];
```

```
        }
```

```
    }
```

```
    printf("the second smallest from array is %d", s2);
```

```
    return 0;
```

```
}
```

```
}
```

output :-

Enter number of elements in an array 4

4

1

2

3

4

The second smallest element among all elements in array is 2

Q2) WAP in C to find sum of left diagonal elements in a matrix.

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Enter no. of row in the square matrix");
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[n][n];
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<n; j++) {
```

```
            scanf("%d", &arr[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    int s=0;
```

```
    for (int i=0; i<n; i++) {
```

```
        s = s + arr[i][i];
```

```
    }
```

```
    printf("\n");
```

```
    printf("The sum of elements of left diagonal is %d", s);
```

```
    return 0;
```

```
}
```

output

enter no of ~~elements~~ rows in the square matrix 2

enter elements of matrix

2
9
2
2

$$\begin{bmatrix} 2 & 9 \\ 2 & 2 \end{bmatrix}$$

the sum of elements on left diagonal is 4.

Q3) WAP in C to find the sum of rows and columns of matrix.

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("enter number of rows in the square matrix");
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[n][n];
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<n; j++) {
```

```
            scanf("%d", &arr[i][j]);
```

```
        }
```

```
        printf("%d\n", arr[i][j]);
```

```
    }
```

```
    int r[n], c[n];
```

```
    printf("the given matrix is");
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<n; j++) {
```

```
            printf("%d ", arr[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```



```
int s1, s2 = 0;
```

```
for (int i=0; i<n; i++) {  
    for (int j=0; j<n; j++) {  
        s1 = s1 + arr[i][j];  
        s2 = s2 + arr[j][i];  
    }  
    s[i] = s1;  
    s1 = 0;  
    c[i] = s2;  
    s2 = 0;  
}
```

```
printf("row sums");  
for (int i=0; i<n; i++) {  
    printf("%d", r[i]);  
}; printf("\n");  
printf("column sums");  
for (int i=0; i<n; i++) {  
    printf("%d", c[i]);  
};  
}
```

output

enter no. of rows in square matrix 2

1
2
1
2

$$\begin{bmatrix} 1 & 2 \\ 1 & 2 \end{bmatrix}$$

your matrix is

1	2
1	2

row sums:

3

3

column sums

2

4

Qn) write a program in c to count the total number of duplicate elements in an array.

```
int main(){
```

```
    printf("enter number of elements in array");
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[100];
```

```
    for (int i=0; i<n; i++){
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    qsort(arr, n, sizeof(int), compare_integers);
```

```
    int count = 0;
```

```
    for (int i=1; i<n; i++){
```

```
        if (arr[i] == arr[i-1]){
```

```
            count++
```

```
            while (i+1 < n && arr[i] == arr[i+1]){
```

```
                i++;
```

```
            }
```

```
        }
```

```
    }
```

```
    printf("the total no. of duplicate elements are %d",
```

```
    count);
```

```
}
```

output

enter no. of elment in array 5

5

5

2

1

3

The total no. of duplicate elements, are 1.

Q5) write a program in C to find the second largest element in an array.

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("enter the number of elements in the array");
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[20];
```

```
    for (int i=0; i<n; i++){
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int b1 = -1000;
```

```
    int b2 = -1000;
```

```
    for (int j=0; j<n; j++){
```

```
        if (arr[j] > b1){
```

```
            b2 = b1;
```

```
            b1 = arr[j];
```

```
        }
```

```
        else if (b2 < arr[j] && b1 != arr[j]){
```

```
            b2 = arr[j];
```

```
        }
```

```
    }
```

```
    printf("the second smallest element from the array is %d", b2);
```

output

enter no. of elements in array 5

1

2

3

4

5

The second smallest element from the array is 2

Q6) WAP in c to delete an element at a desired position from an array.

#include <stdio.h>

int main()

printf("enter number of element in array");

int n;

scanf("%d", &~~arr~~ⁿ);

int arr[20];

for (int i=0; i<n; i++){

scanf("%d", &arr[i]);

}

int pos;

printf("enter pos of element to delete");

scanf("%d", &pos);

if (pos < 0 || pos > n){

printf("invalid pos");

}

for (int j=pos; j<n-1; j++){

arr[pos] = arr[pos+1];

}

printf("the new array is");

for (int i=0; i<n-1; i++){

printf("%d", arr[i]);

}

return 0;

}

output

enter number of element in array 6

2

3

4

5

6

7

enter pos of element to delete 1

the new array

~~2~~

~~3~~

2
4
5
6
7

Week-1

Q) WAP in C to ~~the~~ simulate CPU scheduling algorithm.

1. FCFS

2. SJF.

1) #include <stdio.h>

int main() {

int n, i, j;

int p[10], AT[10], BT[10], TAT[10], CT[10], WT[10];

float avg-wt = 0, avg-tat = 0;

printf("Enter the number of processes");

scanf("%d", &n);

printf("Enter the arrival time & Burst time p:\n");

for (i = 0; i < n; i++) {

printf("p%d Arrival time: ", i+1);

scanf("%d", &AT[i]);

printf("p%d Burst time: ", i+1);

scanf("%d", &BT[i]);

p[i] = i+1;

for (i = 0; i < n-1; i++) {

for (j = i+1; j < n; j++) {

if (AT[i] > AT[j]) {

swap(&AT[i], &AT[j]);

swap(&BT[i], &BT[j]);

swap(&p[i], &p[j]);

}

} Sorting

$CT[i][j] = AT[i][j] + BT[i][j]$

for ($i=1; i \leq n; i++$) {

if ($CT[i-1][j] < AT[i][j]$) {

$CT[i][j] = AT[i][j] + BT[i][j];$

else {

$CT[i][j] = CT[i-1][j] + BT[i][j];$

}

}

for ($i=0; i \leq n; i++$) {

$TAT[i][j] = CT[i][j] - AT[i][j];$

$WT[i][j] = TAT[i][j] + BT[i][j];$

$avg_wt += WT[i][j];$

$avg_tat += TAT[i][j];$

}

$avg_wt /= n;$

$avg_tat /= n;$

printf ("Process 1: AT \n BT \n CT \n TAT \n WT \n RT");

for ($i=0; i \leq n; i++$) {

printf ("P%d \t %d \t %d \t %d \t %d \t %d",

$P[i], AT[i], BT[i], CT[i],$
 $TAT[i], WT[i]$);

}

printf ("Avg WT: %d", avg_wt);

printf ("Avg TAT: %d", avg_TAT);

}

void swap(int *a, int *b) {

int temp = *a;

*a = *b;

*b = temp;

}

Output

Enter the number of processes:

Enter arrival time and burst time:

P1 AT: 0

P1 BT: 3

P2 AT: 4

P2 BT: 3

P3 AT: 6

P3 BT: 2

Process	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P2	4	3	7	3	0
P3	6	2	9	3	1

Sum:

$$\frac{9}{3} = 3$$

$$\frac{1}{3} = 0.33$$

avg wt: 0.33

avg ~~tat~~ tat: 3.00

2) #include <stdio.h>

```
void swap (int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int main() {
```

```
    int n, i, j, choice;
```

```
    int p[20], at[20], bt[20], ct[20], tat[20], wt[20];
```

```

float avg-ut = 0, avg-tat = 0;
printf("Enter number of Process");
scanf("%d", &n);
printf("Enter at & at");
for (int i=0; i<n; i++) {
    printf("P% Aet :", i+1);
    scanf("%d", &AT[i]);
    printf("P% Bt :", i+1);
    scanf("%d", &BT[i]);
    P[i]
    P[i] = i+1;
    RT[i] = BT[i];
}

```

```

printf("Choose Scheduling Type\n");

```

```

printf("1. non preemptive SJF");

```

```

printf("2. preemptive SJF");

```

```

printf("Enter choice");

```

```

scanf("%d", &choice);

```

```

switch (choice) {

```

```

    case 1: {

```

```

        int time = 0, completed[20] = {0}, count = 0;

```

```

        while (count < n) {

```

```

            int idx = -1, min_bt = 999;

```

```

            for (i=0; i<n; i++) {

```

```

                if (!completed[i] && AT[i] <= time
                    && BT[i] < min_bt) {

```

```

                    min_bt = BT[i];

```

```

                    idx = i;

```

```

                }

```

```

            }

```



```
if (idx == -1)
```

```
time ++;
```

```
}
```

```
else {
```

```
time += BT[idx];
```

```
CT[idx] = time;
```

```
completed[idx] = 1;
```

```
count ++;
```

```
}
```

```
}
```

```
break;
```

```
}
```

```
Case 2: {
```

```
int time = 0, completed = 0;
```

```
while (completed < n) {
```

```
int idx = -1, min_rt = 9999;
```

```
for (i = 0; i < n; i++) {
```

```
if (AT[i] <= time && RT[i] > 0 &&
```

```
RT[i] < min_rt) {
```

```
min_rt = RT[i];
```

```
idx = i;
```

```
}
```

```
}
```

```
if (idx == -1) {
```

```
time ++;
```

```
}
```

```
else {
```

```
RT[idx] --;
```

```
time ++;
```

```
if (RT[idx] == 0) {
```

```
CT[idx] = time;
```

```
completed ++;
```

```
}
```

```
}
```

```
}
```

break;

}

default:

printf("Invalid choice!\n");
return 0;

}

for (i=0; i<n; i++)

TAT[i] = CT[i] - AT[i];

WT[i] = TAT[i] - BT[i];

avg_wt += WT[i];

avg_tat += ~~WT~~ TAT[i];

}

avg_wt /= n;

~~avg~~ avg_wt /= n;

avg_tat /= n;

printf("Process AT BT CT TAT WT RT")

, ~~RT~~

for (i=0; i<n; i++)

printf("P%d %d %d %d %d %d %d",

P[i], AT[i], BT[i], CT[i], TAT[i], WT[i],
RT[i]);

}

printf("avg wt: %.2f", avg_wt);

printf("avg TAT: %.2f", avg_tat);

printf("avg RT: %.2f", avg_RT);

~~return 0;~~

}

Output

Enter number of Process : 4

Enter arrival time and burst time for each Process

P₁ AT = 0

P₁ BT = 5

P₂ AT = 1

P₂ BT = 3

P₃ AT = 2

P₃ BT = 8

P₄ AT = 3

P₄ BT = 6

Choose scheduling type:

1. Non preemptive
2. preemptive

1

Process	AT	BT	WT	TAT
P ₁	0	5	3	8
P ₂	1	3	0	3
P ₃	2	8	12	20
P ₄	3	6	5	11

Avg waiting time : 5

avg TAT : 10.5

By comparing all SJF non preemptive has shortest WT.

Week-2

Priority (non-preemptive)

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("enter number of process");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], ct[n], tat[n], wt[n], p[n], pri[n];
```

```
    int completed[n];
```

```
    for (int i=0; i<n; i++) {
```

```
        printf("enter At of P %d in ", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("enter Bt of P %d in ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
printf("enter Pt of P %d in ", i+1);
```

```
        printf("enter Pt of P %d in ", i+1);
```

```
        scanf("%d", &pt[i]);
```

```
        p[i] = i+1;
```

```
        completed[i] = 0;
```

```
    }
```

```
    int total = 0, time = 0;
```

```
    float avg_tat, avg_wt;
```

```
    int highp = 999, idx;
```

```
    while (total < n) {
```

```
        highp = 999;
```

```
        idx = -1;
```

```
for (int i=0; i<n; i++) {
```

```
    if (at[i] <= time && p[i] >= highp && !completed[i])
```

```
        highp = p[i];
```

```
        idx = i;
```

```
    }
```

```
    if (idx == -1) {
```

```
        time = time + 1;
```

```
        continue;
```

```
    }
```

```
    time = time + bt[idx];
```

```
    completed[idx] = 1;
```

```
    ct[idx] = time;
```

```
    total = total + 1;
```

```
}
```

```
for (int i=0; i<n; i++) {
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
    avg_tat = avg_tat + tat[i];
```

```
    avg_wt = avg_wt + wt[i];
```

```
}
```

```
printf("Imp tat \t Bt \t PRI \t CT \t TAT \t wt \n");
```

```
for (int i=0; i<n; i++)
```

```
    tat[i] = ct[i] + at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
    printf ("p %d it %d it %d it %d it %d it %d it", i+1, at[i],
            bt[i], ct[i], tat[i], wt[i]);
```

```
}
```

```
avg_wt = avg_wt/n;
```

```
avg_tat = avg_tat/n;
```

```
printf ("the average wt: %f\n", avg_wt);
```

```
printf ("the average tat: %f\n", avg_tat);
```

```
return 0;
```

```
}
```

output:

enter number of process: 3

P1 at: 0

BT: 4

Pri: 2

P2 at: 3

BT: 3

Pri: 1

P3 at: 4

BT: 3

Pri: 3

process	AT	BT	Pri	CT	TAT	WT
1	0	4	2	10	16	6
2	3	3	1	6	3	0
3	4	3	3	13	7	6

$$\text{Avg TAT} = \frac{27}{3} = 9$$

$$\text{Avg wt} = \frac{12}{3} = 4$$

* preemptive

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
```

```
    int bt[10], at[10], pr[10];
```

```
    int rt[10], ct[10], tct[10], wt[10];
```

```
    int highest, min-pr;
```

```
    printf("enter number of process:");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("process %d is ", i+1);
```

```
        printf("AT:");
```

```
        scanf("%d", &at[i]);
```

```
        printf("priority");
```

```
        scanf("%d", &pr[i]);
```

```
        rt[i] = bt[i];
```

```
    }
```

```
    while (completed != n) {
```

```
        highest = -1;
```

```
        min-pr = 999;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (at[i] <= time && rt[i] > 0 && pr[i] < min-pr) {
```

```
                min-pr = pr[i];
```

```
                highest = i;
```

if (highest == -1)

time ++;

continue;

}

rt[highest] --;

time ++;

if (rt[highest] == 0)

completed ++;

at[highest] = time;

bt[highest] = ct[highest] - at[highest];

}

}

printf ("In Process \t A \t B \t C \t A \t (WT \t Z)\n",
i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);

}

output

Enter number of processes : 3

P1

AT : 0

BT : 4

P : 2

P2

AT : 3

BT : 3

P : 1

P3

AT : 4

BT : 3

P : 3

Process	AT	BT	PCT	Pri	TAT	WT
1	0	4	7	2	7	3
2	3	3	6	1	3	0
3	4	3	10	3	6	3

Avg TAT = 5.34

Avg wt = 2.



```
printf("Burst time");  
scanf("%d", &P[i].bt);  
P[i].rt = P[i].bt;
```

```
}
```

```
printf("\n Enter time quantum: ");
```

```
scanf("%d", &tq);
```

```
timeMark[0] = 0;
```

```
while (Completed < n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (P[i].rt <= time && visited[i] == 0) {
```

```
            queue[rear++] = i;
```

```
            visited[i] = 1;
```

```
        }
```

```
    if (front == rear) {
```

```
        time++;
```

```
        continue;
```

```
    }
```

```
    int idx = queue[front++];
```

```
    order[k] = P[idx] - P[id];
```

```
    if (P[idx].rt > tq) {
```

```
        P[idx].rt -= tq;
```

```
        time += tq;
```

```
    }
```

```
    else {
```

```
        time += P[idx].rt;
```

```
        P[idx].rt = 0;
```

```
        P[idx].rt = time;
```

1. The first part of the paper is devoted to a discussion of the

theoretical background of the problem.

2. The second part is devoted to a discussion of the

experimental results.

3. The third part is devoted to a discussion of the

conclusions.

4. The fourth part is devoted to a discussion of the

conclusions.

5. The fifth part is devoted to a discussion of the

conclusions.

6. The sixth part is devoted to a discussion of the

conclusions.

7. The seventh part is devoted to a discussion of the

conclusions.

8. The eighth part is devoted to a discussion of the

conclusions.

9. The ninth part is devoted to a discussion of the

conclusions.

10. The tenth part is devoted to a discussion of the

conclusions.

11. The eleventh part is devoted to a discussion of the

```
printf ("Process %d AT %d BT %d CT %d TAT %d WT %d\n",
```

```
for (int i=0; i<n; i++)
```

```
printf ("P%d %d %d %d %d %d %d\n",
```

```
P[i], P[i].id, P[i].at, P[i].bt, P[i].tat,
```

```
P[i].wt);
```

```
}
```

```
printf ("In Average TAT = %.2f", avg_tat/n);
```

```
printf ("In Average WT = %.2f", avg_wt/n);
```

```
return 0;
```

```
}
```

Output

Enter number of process: 3

P1

AT: 0

BT: 7

P2

AT: 3

BT: 5

P3

AT: 6

BT: 11

Enter TQ: 5

If TQ: 6

avg TAT = 14.33

avg WT = 6.67

27/3

Analysis

while the Time quantum is increased under the average B.T the waiting time and TAT will reduced.

Process	AT	BT	CT	TAT	WT
P1	0	7	12	12	5
P2	3	5	10	7	2
P3	6	11	23	17	6

avg TAT = 12.00

avg WT = 4.33

For same Arrival time

tq = 2

Aug TAT = 16.67

Aug WT = 10

tq = 3

Aug TAT = 16

Aug WT = 9.33

Q) write a c Program to simulate real-time CPU scheduling algorithm:

a) Rate-Monotonic

b) Earliest deadline first

c) Proportional scheduling.

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define MAX 20
```

```
#define MAX_TIME 1000
```

```
int gcd(int a, int b) {
```

```
if (b == 0) return a;
```

```
return gcd(b, a % b);
```

```
}
```

```
int
```

```
struct Task {
```

```
int id;
```

```
int bt;
```

```
int deadline;
```

```
int period;
```

```
int ct;
```

```
int wt;
```

```
int tat;
```

```
};
```

```
int gcd (int a, int b) {
```

```
if (b == 0) return a;
```

```
return gcd (b, a % b);
```

```
}
```

```
int lcm (int a, int b) {
```

```
return (a * b) / gcd (a, b);
```

```
}
```

```

int find-hyperperiod (struct Task tasks [], int n) {
    int res = tasks[0].period;
    for (int i=1; i<n; i++) {
        res = lcm(res, tasks[i].period);
    }
    return res;
}

```

```

double utilization-edf (struct Task tasks[], int n) {
    double u=0;
    for (int i=0; i<n; i++) {
        u += (double) tasks[i].bt / tasks[i].period;
    }
    return u;
}

```

```

double rm-bound (int n) {
    return n * (pow(2.0, 1.0/n) - 1.0);
}

```

```

void print-chart (int schedule [], int total, char
    title []) {
    printf ("m%s Grantt chart: m", title);

    for (int i=0; i<total; i++) {
        if (schedule[i] == 0)
            printf ("idle |");
        else
            printf ("p:%d |", schedule[i]);
    }
    printf ("\n");

    for (int i=1; i<=total; i++) {
        printf ("%5d", i);
    }
    printf ("\n");
}

```

```

void edf (struct Task tasks[], int n) {
    int time = 0;
    int schedule[MAX_TIME] = {0};
    int remaining[MAX], next_release[MAX],
        deadline[MAX];
    for (int i = 0; i < n; i++) {
        remaining[i] = 0;
        next_release[i] = 0;
        deadline[i] = tasks[i].deadline;
        tasks[i].ct = tasks[i].wt = tasks[i].tat = 0;
    }
    int total_time = find_hiperiod(tasks, n);
    printf("\n\n == Earliest Deadline First == \n\n");
    while (time < total_time) {
        int selected = -1;
        int min_deadline = 99999;
        for (int i = 0; i < n; i++) {
            if (time == next_release[i]) {
                remaining[i] = tasks[i].bt;
                deadline[i] = time + tasks[i].deadline;
                next_release[i] += tasks[i].period;
            }
            if (remaining[i] > 0 && deadline[i] < min_deadline) {
                min_deadline = deadline[i];
                selected = i;
            }
        }
    }
}

```

```

if (selected != -1) {
    schedule[time] = tasks[selected];
    remaining[selected]--;
}
if (remaining[selected] == 0) {
    tasks[selected].ct++;
    tasks[selected].tat++;
    tasks[selected].wt++;
}
}
else {
    schedule[time] = 0;
    time++;
}
printf("\n process \t id \t wt \t ct \n");
for (int i = 0; i < n; i++) {
    printf("P %d \t %d \t %d \t %d \n",
        tasks[i].id,
        tasks[i].ct,
        tasks[i].wt,
        tasks[i].tat);
}
printf("\n\n chart (schedule)\n\n");
double u = u u;
double bound = 1;
if (u <= bound) {
    printf("RMS\n");
}
}

```

```
if (selected != -1) {
```

```
    schedule[time] = tasks[selected].id;
```

```
    remaining[selected] --;
```

```
    if (remaining[selected] == 0) {
```

```
        tasks[selected].ct = time + 1;
```

```
        tasks[selected].tat = tasks[selected].ct;
```

```
        tasks[selected].wt = tasks[selected].tat;
```

```
        tasks[selected].wt = tasks[selected].tat -  
            tasks[selected].bt;
```

```
    }
```

```
}
```

```
else {
```

```
    schedule[time] = 0;
```

```
}
```

```
time ++;
```

```
}  
printf ("In process \t BT \t Period \t ct \t wt \t tat \n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf ("P %d \t %d \t %d \t %d \n",
```

```
        tasks[i].id, tasks[i].bt, tasks[i].period,
```

```
        tasks[i].ct, tasks[i].wt, tasks[i].tat);
```

```
}
```

```
Print_RMS_Chart (schedule, time, "RMS");
```

```
double u = utilization find_utilization (tasks, n);
```

```
double bound = rm_bound (n);
```

```
if (u <= bound) {
```

```
    printf ("RMS result : Sched fu = %.2f, Bound = %.2f  
            \n", u, bound);
```

```
}
```


else {

```
printf ("Rms result: Not guaranteed (U = %.2f ,  
        Bound = %.2f) \n", u, bound);
```

}

```
int main() {
```

```
    int n;
```

```
    struct Tasks tasks[100];
```

```
    printf ("Enter number of process");
```

```
    scanf ("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        tasks[i].id = i + 1;
```

```
        printf ("Process %d\n", i + 1);
```

```
        printf ("Burst time:");
```

```
        scanf ("%d", &tasks[i].bt);
```

```
        printf ("Deadline (for EDF):");
```

```
        scanf ("%d", &tasks[i].deadline);
```

```
        printf ("Period (for Rms):");
```

```
        scanf ("%d", &tasks[i].period);
```

```
    }
```

```
    edf (tasks, n);
```

```
    rms (tasks, n);
```

```
    return 0;
```

```
}
```

output

enter number of processes: 3

Process 1

Burst time: 3

Deadline (for EDF) = 7

Period (for RMS) = 20

Process 2

Burst time: 2

Deadline (for EDF) = 4

Period (for RMS) = 5

Process 3

Burst time: 2

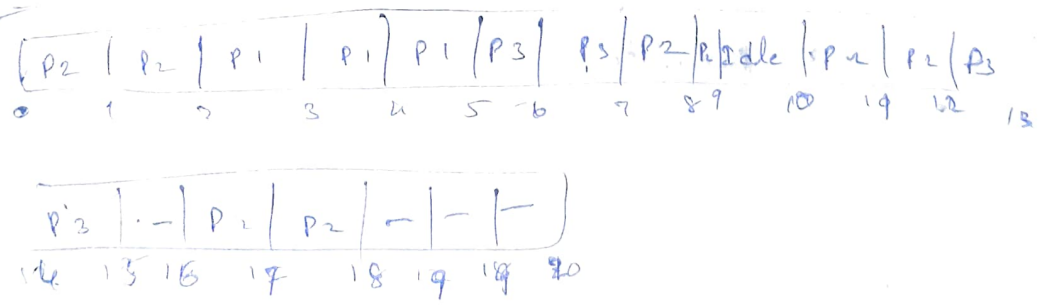
Deadline (for EDF) = 8

period (for RMS) = 10

Earliest Deadline first (EDF) -

Process	BT	Deadline	ET	WT	TAT
P1	3	7	5	2	5
P2	2	4	17	15	17
P3	2	8	14	12	14

EDF Gantt chart

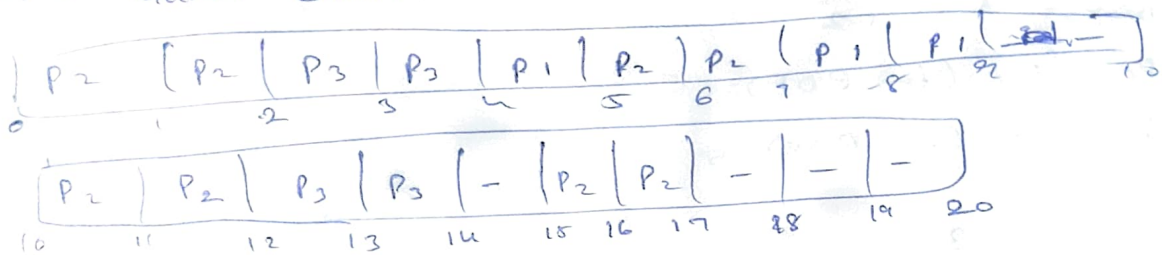


EDU result : schedulable (U = 0.75).

Rate Monotonic scheduling (RMS)

Process	BT	period	CT	WT	TAT
P ₁	3	20	9	6	9
P ₂	2	5	17	15	17
P ₃	2	10	14	12	14

RMS Gantt chart:



RMS Result: Schedulable ($U = 0.75$, Bound 20.78)

Proportionality scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct process {
    int pid;
    int bursts;
    int remaining;
    int completed;
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    printf("enter number of process : ");
```

```
    scanf("%d", &n);
```

```
struct process P[n];
```

```
for (i=0 ; i<n ; i++) {
```

```
    P[i].pid = i+1;
```

```
    P[i].completed = 0;
```

```
    printf("enter burst time and tickets for  
        P%d : ", i+1);
```

```
    scanf ("%d %d", &P[i].burst, &P[i].tickets);
```

```
    P[i].remaining = P[i].burst;
```

```
}
```

```
 srand (time(0));
```

```
int time = 0; done = 0;
```

```
printf ["in Execution order, \n");
```

```
while (done < n) {
```

```
    int totaltickets = 0;
```

```
    for (i=0; i<n; i++) {
```

```
        if (!P[i].completed)
```

```
            totaltickets += P[i].tickets;
```

```
    }
```

```
    int winner = rand() % totaltickets + 1;
```

```
    int sum = 0;
```

```
    int selected = -1;
```

```
    for (int i=0; i<n; i++) {
```

```
        if (P[i].completed) {
```

```
            sum += P[i].tickets;
```

```
            if (sum > winner) {
```

```
                selected = i;
```

```
                break;
```

```
            }
```

```
        }
```

```
}
```

Printf ("time %d → process P %d selected",

time, P[selected].pid);

P[selected].remaining--;

time++;

if (P[selected].remaining == 0) {

 P[selected].completed++;

 done++;

 Printf ("Process P %d completed at time %d\n", P[selected].pid, time);

}

}

Printf ("All processes completed\n");

return 0;

}

output

Enter number of process : 3

Enter burst time and tickets for P1 : 5 20

Enter burst time and tickets for P2 : 4 40

Enter burst time and tickets for P3 : 5 60

Execution order

0 → P1 selected

1 → P1 "

2 → P2 "

3 → P2 "

4 → P2 "

~~P2~~ → completed

5 → P₁ selected

6 → P₃ "

7 → P₁ "

8 → P₃ "

9 → P₃ "

10 → P₁ "

11 → P₃ "

12 → P₃ "

P₃ → completed at ~~B~~ B

13 → P₁ selected

~~P1~~ completed

28/4

Week 5

i) Bounded Buffer

ii) Dining philosopher problem

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
```

```
#define N5
```

```
#define EAT_COUNT3
```

```
pthread_mutex_t forks[N];
```

```
void * philosopher(void *arg) {
```

```
    int id = *(int *)arg;
```

```
    for (int i=0; i<EAT_COUNT; i++) {
```

```
        printf("philosopher %d is thinking.\n", id);
```

```
        if (id % 2 == 0) {
```

```
            pthread_mutex_lock(&forks[id]);
```

```
            pthread_mutex_lock(&forks[(id+1)%N]);
```

```
        } else {
```

```
            pthread_mutex_lock(&forks[(id+1)%N]);
```

```
            pthread_mutex_lock(&forks[id]);
```

```
        }
        printf("philosopher %d is eating.\n", id);
```

```
        pthread_mutex_unlock(&forks[id]);
```

```
        pthread_mutex_unlock(&forks[(id+1)%N]);
```

printf ("philosopher %d put down forks %d, %d);

return NULL;

```
3  
int main() {  
    pthread_t philo[N];  
    int ids[N];  
    for (int i=0; i<N; i++) {  
        pthread_mutex_init (&forks[i], NULL);  
        ids[i] = i;  
    }  
    for (int i=0; i<N; i++) {  
        pthread_join (philo[i], NULL);  
    }  
    for (int i=0; i<N; i++) {  
        pthread_mutex_destroy (&forks[i]);  
    }  
    return 0;  
}
```

output

philosopher 0 is thinking
" 0 is eating
" 3 is thinking
3 is eating
3 put down forks
3 is thinking
" is thinking
" is thinking
1 is eating
1 put down forks.

i) Bounded Buffer

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define Buffer-size 5
#define produce-count 10

int buffer[Buffer-size];
int in = 0, out = 0;
int count = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not-full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not-empty = PTHREAD_COND_INITIALIZER;

void * producer(void * arg) {
    for (int i = 0; i < produce-count; i++) {
        pthread_mutex_lock(&mutex);
        while (count == Buffer-size) {
            pthread_cond_wait(&not-full, &mutex);
        }
        buffer[in] = i;
        in = (in + 1) % Buffer-size;
        count++;
        pthread_cond_signal(&not-empty);
        pthread_mutex_unlock(&mutex);
    }
}
```

```
void * consumer (void * arg) {
```

```
    for (int i=0; i < produce_count; i++) {
```

```
        pthread_mutex_lock (&mutex);
```

```
        while (count == 0) {
```

```
            pthread_cond_wait (&not_empty,  
                                &mutex);
```

```
        }
```

```
        int item = buffer[low];
```

```
        printf ("consumed %d from buffer (%d)  
               in", item, out);
```

```
        out = (out+1) % Buffer_size;
```

```
        count--;
```

```
        pthread_cond_signal (&not_full);
```

```
        pthread_mutex_unlock (&mutex);
```

```
    }
```

```
    return NULL;
```

```
}
```

```
int main () {
```

```
    pthread_t prod, cons;
```

```
    pthread_create (&prod, NULL, producer, NULL);
```

```
    pthread_create (&cons, NULL, consumer, NULL);
```

```
    pthread_join (prod, NULL);
```

```
    pthread_join (cons, NULL);
```

```
    return 0;
```

```
}
```

output

produced 0 at buffer[0]
 produced 1 at buffer[1]
 produced 2 at buffer[2]
 produced 3 at buffer[3]
 " u at buffer[4]

consumed 0 from buffer[0]
 " 1 " buffer[1]
 " 2 " buffer[2]
 " 3 " buffer[3]
 " u " buffer[4]

produced 5 at buffer[0].

~~Tracing~~
~~step operation buffer state in~~

step	operation	Buffer state	in	out	empty	full	mutex
1	$P \rightarrow 0$	[0, -, -, -]	1	0	u	1	1
2	$P \rightarrow 1$	[0, 1, -, -]	2	0	3	2	1
3	$P \rightarrow 2$	[0, 1, 2, -]	3	0	2	3	1
4	$P \rightarrow 3$	[0, 1, 2, 3]	4	0	1	4	1
u	$P \rightarrow 4$	[0, 1, 2, 3, u]	0	0	0	5	1
5	$P \rightarrow u$	[0, 1, 2, 3, u]	0	0	0	5	1
6	$P \rightarrow X$	[0, 1, 2, 3, u]	0	0	0	5	1
7	Consumer removes 0	[-, 1, 2, 3, u]	0	1	1	u	1

24/4

Write a C Program to simulate the bankers algorithm and deadlock detection

Bankers algorithm

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
void calculateNeed (int n, int m, int max[n][m],  
int alloc[n][m], int need[n][m]) {
```

```
for (int i=0; i<n; i++)
```

```
for (int j=0; j<m; j++) {
```

```
    need[i][j] = max[i][j] - alloc[i][j];
```

```
}
```

```
}
```

```
bool isSafe (int n, int alloc[n][m], int need[n][m],  
int avail[m]) {
```

```
    bool finish[n];
```

```
    for (int i=0; i<n; i++) finish[i] = false;
```

```
    int safeSeq[m], work[m];
```

```
    for (int j=0; j<m; j++) work[j] = avail[j];
```

```
    int count = 0;
```

```
    while (count < n) {
```

```
        bool found = false;
```

```
        for (int i=0; i<n; i++) {
```

```
            if (!finish[i]) {
```

```
            }
```

```

    ind i;
    for (j=0; j<m; j++) {
        if (need[i][j] > work[j])
            break;
    }
    if (j==m) {
        for (int k=0; k<m; k++) {
            work[k] += alloc[i][k];
        }
        safeSeq[count++] = i;
        finish[i] = true;
        found = true;
    }
}

```

```

}
if (!found) {
    printf("The system is not a safe state.\n");
    return false;
}

```

```

printf("The system is in safe state: safe seq is: ");
for (int i=0; i<n; i++) {
    printf("%d ", safeSeq[i]);
}
if (i==n-1) printf("\n");

```

```

return true;
}

```

```
bool requestResources (int process, int m, int n,  
    int request[m], int alloc[m],  
    int need[n][m], int avail[m]) {
```

```
    for (int i=0; i<n; i++) {
```

```
        if (request[i] > need[process][i]) {
```

```
            printf ("Error: process has exceed  
                max claim");
```

```
            return false;
```

```
        }
```

```
    }
```

```
    for (int i=0; i<m; i++) {
```

```
        if (request[i] > avail[i]) {
```

```
            printf ("Resource not av. Process waiting");
```

```
            return false;
```

```
        }
```

```
    }
```

```
    for (int i=0; i<m; i++) {
```

```
        avail[i] -= request[i];
```

```
        alloc[process][i] += request[i];
```

```
        need[process][i] -= request[i];
```

```
    }
```

```
    if (isSafe (n, m, alloc, need, avail))
```

```
        return true;
```

```
    else {
```

```
        for (int i=0; i<m; i++) {
```

```
            avail[i] += request[i];
```

```
            alloc[process][i] -= request[i];
```

```
            need[process][i] += request[i];
```

```
        }
```

return false;

~~if (isSafe(n, m, alloc, need, avail))~~
~~return true~~

int main (C) {

int n, m;

printf ("enter number of process");

scanf ("%d", &n);

printf ("enter number of res");

scanf ("%d", &m);

int alloc[n][m], max[n][m], avail[m], need[n][m];

printf ("enter allocation matrix");

for (int i=0; i<n; i++)

for (int j=0; j<m; j++)

scanf ("%d", &alloc[i][j]);

printf ("enter Max demand matrix");

for (int i=0; i<n; i++)

for (int j=0; j<m; j++)

scanf ("%d", &Max[i][j]);

printf ("enter available Resources");

for (int i=0; i<m; i++)

scanf ("%d", &avail[i]);

calculateNeed (n, m, max, alloc, need);

~~if~~

isSafe (n, m, alloc, need, avail);

int * request = malloc (n * sizeof (int));

requestResource (1, n, m, request, alloc, need, avail);

return 0;

adpt

enter no. of process : 5

enter n of resources : 3

enter allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

enter Max demand matrix

7 5 3

3 2 2

9 0 2

2 2 2

6 3 3

enter available resources

3 3 2

System is in a Safe state

Safe seq is : P1 P3 P4 P0 P2

deadlock detection

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
void calculateNeed (int n, int m, int max[n][m],  
int alloc[n][m], int need[n][m])
```

```
for (int i=0; i<n; i++)
```

```
for (int j=0; j<m; j++)
```

```
need[i][j] = max[i][j] - alloc[i][j];
```

```
}
```



```
void detectDeadlock (int n, int m, int alloc[n][m], int  
need[n][m], int avail[m]) {
```

```
    bool finish[n];
```

```
    for (int i=0; i<n; i++)
```

```
        finish[i] = false;
```

```
    int work[m];
```

```
    for (int j=0; j<m; j++) {
```

```
        work[j] = avail[j];
```

```
    }
```

```
    int count = 0;
```

```
    while (count < n) {
```

```
        bool found = false;
```

```
        for (int i=0; i<n; i++) {
```

```
            if (!finish[i]) {
```

```
                int j;
```

```
                for (j=0; j<m; j++)
```

```
                    if (need[i][j] > work[j])
```

```
                        break;
```

```
                if (j == m) {
```

```
                    for (int k=0; k<m; k++) {
```

```
                        work[k] += alloc[i][k];
```

```
                        finish[i] = true;
```

```
                        found = true;
```

```
                        count++;
```

```
                    }
```

```
                }
```

```
            }
```

```

bool deadlock = false;
int deadlocked_procs[n], deadlocked_count = 0;
for (int i = 0; i < n; i++) {
    if (!finish[i]) {
        deadlock = true;
        deadlocked_procs[deadlocked_count++] = i;
    }
}

```

```

if (!deadlock) {
    printf("no dead deadlock detected\n");
}
else {
    printf("Deadlocked processes:");
    for (int i = 0; i < deadlocked_count; i++) {
        printf("P%d", deadlocked_procs[i]);
        if (i != deadlocked_count - 1)
            printf(", ");
    }
    printf("\n");
}

```

```

}
printf("\n");

```

```

}

```

```

int main() {
    int n, m;
    printf("enter number of process");
    scanf("%d", &n);
    printf("enter resources");
    scanf("%d", &m);
}

```

```

int alloc[5][3], max[5][3], avail[3], need[5][3];
printf("enter allocation matrix");
for (int i=0; i<n; i++)
    for (int j=0; j<m; j++)
        scanf("%d", &alloc[i][j]);

printf("enter max demand matrix");
for (int i=0; i<n; i++)
    for (int j=0; j<m; j++)
        scanf("%d", &max[i][j]);

printf("enter available resources");
for (int i=0; i<m; i++)
    scanf("%d", &avail[i]);

calculate(n, m, max, alloc, need);
printf("Deadlock detection");
detect_deadlock(n, m, alloc, need, avail);
return 0;

```

Ans:

enter no. of process 5
 enter no. of resources 3
 enter allocation matrix

0	0
2	0
3	0
2	0
0	0

enter max demand matrix

7	5
3	2
9	0
2	2
6	3

enter available resources
 3 2 2

No deadlock detected.

Q) Write a C program to simulate the following configuration memory allocation techniques.

- Worst-fit
- Best-fit
- First-fit

Given a free memory partition: 400k, 700k, 200k, 300k, 600k. How would each of 1st, Best and worst algorithm place the offers of 212k, 512k, 112k, 526k

First Fit →

212k	512k			312k
400k	700k	200k	300k	600k

Best fit →

312k	526		212	512
200	600	300	200	125

worst fit →

	500	200		115	
300	600	350	200	150	125

code

```
#include <stdio.h>
```

```
void firstFit (int blockSize[], int m, int process[], int n) {
```

```
int allocation[n];
```

```
for (int i=0; i<n; i++) allocation[i] = -1;
```

```
for (int i=0; i<n; i++) {
```

```
for (int j=0; j<m; j++) {
```

```
if (blockSize[j] >= process[i]) {
```

```

allocation[i] = 1;
}
else if (blocksize[i] == -1) {
    break;
}
}
}

```

```

printf("\n --- First Fit ---");
printf("Process No | Process Size | Block No\n");
for (int i = 0; i < n; i++) {
    if (allocation[i] != -1) {
        printf("%d | %d | %d\n", i+1,
            process[i],
            allocation[i]);
    }
    else {
        printf("%d | %d | Not Allocated\n", i+1, process[i]);
    }
}

```

```

void firstFit(int blocksize[], int n, int processsize[], int m) {
    int allocation[m];
    for (int i = 0; i < m; i++) allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blocksize[j] >= processsize[i]) {
                allocation[j] = i;
                break;
            }
        }
    }
}

```



```

if (bestidx != -1) {
    allocation[i] = bestidx;
    blocksize[bestidx] = -1;
}

```

```

printf ("In Best fit ");
printf ("Process no. \t Process size \t Block No. \n");
for (int i = 0; i < n; i++) {
    if (allocation[i] != -1)
        printf ("%d \t \t %d \t \t %d", i+1, ProcessSize[i],
            allocation[i]);
    else
        printf ("%d \t \t %d \t \t Not Allocated \n",
            i+1, ProcessSize[i]);
}

```

```

void worstfit (int blocksize[], int n, int ProcessSize[],
    int n) {

```

```

    int allocation[n];
    for (int i = 0; i < n; i++)
        allocation[i] = -1;
    for (int i = 0; i < n; i++) {
        int worstidx = -1;
        for (int j = 0; j < m; j++) {
            if (worstidx
                blocksize[j] >= ProcessSize[i]) {
                    if (worstidx == -1 || blocksize[j]
                        > blocksize[worstidx])

```



```

printf ("Enter size of %d processes: ", n);
for (int i=0; i<n; i++) {
    scanf ("%d", &processSize[i]);

    int blockCopy1[m], blockCopy2[m], blockCopy3[m];
    for (int i=0; i<m; i++) {
        scanf ("%d", &blockSize[i]);

        int blockCopy1[m], blockCopy2[m], blockCopy3[m];
        for (int i=0; i<m; i++) {
            blockCopy1[i] = blockSize[i];
            blockCopy2[i] = blockSize[i];
            blockCopy3[i] = blockSize[i];
        }
        firstFit (blockCopy1, m, processSize, n);
        bestFit (blockCopy2, m, processSize, n);
        worstFit (blockCopy3, m, processSize, n);
        return 0;
    }
}

```

Output

Enter process: 5

Enter size of 5 processes = 115 300 358 200 375

<u>First fit</u>			<u>Best fit</u>			<u>Worst fit</u>		
Pno	Psize	Block	Pno	Psize	Block	Pno	Psize	Block
1	115	1	1	115	6	1	115	5
2	300	2	2	500	2	2	500	2
3	358	5	3	358	5	3	358	NA
4	200	3	4	200	4	4	200	3
5	375	NA	5	375	NA	5	375	NA

week-8

write a c program to simulate page replacement algorithm

a) FIFO

b) LRU

c) Optimal

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void printFrames (int frame[], int m) {
```

```
    for (int i=0; i<m; i++) {
```

```
        if (frame[i] == -1) printf("- ");
```

```
        else printf("%d ", frame[i]);
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
int findOptimal (int pages[], int n, int frame, int m,  
                int index) {
```

```
    int farthest = index, pos = -1;
```

```
    for (int i=0; i<m; i++) {
```

```
        int j;
```

```
        for (j=index; j<n; j++) {
```

```
            if (frame[i] == pages[j]) {
```

```
                if (j > farthest) {
```

```
                    farthest = j;
```

```
                    pos = j;
```

```
                }  
                break;
```

```
            }
```

```
if (j == -1) return i;
```

```
}
```

```
return (Pos == -1) ? 0 : Pos;
```

```
}
```

```
void fifo (int pages[], int n, int m) {
```

```
    int frame[m], count = 0, pagesFaults = 0;
```

```
    for (int i = 0; i < m; i++) frame[i] = -1;
```

```
    printf ("m --- FIFO Simulation --- n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        int flag = 0;
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (frame[j] == pages[i]) {
```

```
                flag = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if (!flag) {
```

```
            frame[count] = pages[i];
```

```
            count = (count + 1) % m;
```

```
            pagesFaults++;
```

```
        }
```

```
        printf ("pages %d → ", pages[i]);
```

```
        printf ("Total FIFO Pages Faults: %d\n",
```

```
            pagesFaults);
```

```
}
```

```

void LRU_Clock(Pages [n], int n, int m) {
    int frame [n]; lastused [n], Page Faults = 0;
    for (int i=0; i<n; i++) {
        frame[i] = -1;
        lastused[i] = -1;
    }
}

```

```

    printf("\n -- LRU simulation -- \n");

```

```

    for (int i=0; i<n; i++) {
        int flag = 0;
        for (int j=0; j<m; j++) {
            if (frame[i] == Pages[j]) {
                flag = 1;
                lastused[j] = i;
                break;
            }
        }
    }

```

```

    if (!flag) {

```

```

        int min = 0;

```

```

        for (int j=1; j<m; j++) {

```

```

            if (frame[j] == -1) { min = j; break; }

```

```

            if (lastused[j] < lastused[min])

```

```

                min = j;

```

```

        frame[min] = Pages[i];

```

```

        lastused[min] = i;

```

```

        Page Faults ++;
    }
}

```

```
printf ("Page f.d → ", pages[i]);
```

```
printf frames (frame, n);
```

```
}
```

```
printf ("Total LRU pages faults: %d\n", pageFaults);
```

```
}
```

```
void optimal (int pages[], int n, int m) {
```

```
    int frame[m], pageFaults = 0;
```

```
    for (int i = 0; i < m; i++) frame[i] = -1;
```

```
    printf ("In optimal Simulation");
```

```
    for (int i = 0; i < n; i++) {
```

```
        int flag = 0;
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (frame[j] == pages[i]) {
```

```
                flag = 1;
```

```
                break;
```

```
            }
```

```
        }
```

```
        If (!flag) {
```

```
            int pos = -1;
```

```
            for (int j = 0; j < m; j++) {
```

```
                if (frame[j] == -1) { pos = j; break; }
```

```
            }
```

```
            if (pos == -1) pos = findOptimal (pages, n, frame,
```

```
                , m, i+1);
```

```
            frame[pos] = pages[i];
```

```
            pageFaults++;
```

```
        }
```

```
printf ("Page %d → ", pages[i]);
```

```
printf Frames (frames, m);
```

```
}
```

```
printf ("Total optimal page Results: %d\n", pages[i]);
```

```
}
```

```
int main() {
```

```
int n, m, choice;
```

```
printf ("Enter number pages: ");
```

```
scanf ("%d", &n);
```

```
int pages [n];
```

```
printf ("enter the reference string: ");
```

```
for (int i=0; i<n; i++) scanf ("%d", &pages[i]);
```

```
printf ("enter number of frames: ");
```

```
scanf ("%d", &m);
```

```
printf ("In choose Alg ");
```

```
printf ("1. FIFO & 2. LRU & 3. Optimal ");
```

```
scanf ("%d", &choice);
```

```
switch (choice) {
```

```
case 1: fifo (pages, n, m); break;
```

```
case 2: lru (pages, n, m); break;
```

```
case 3: optimal (pages, n, m) break;
```

```
default: printf ("Invalid choice!\n");
```

```
}
```

```
return 0;
```

```
}
```

output

enter no of Pages : 19

enter the pg reference string :

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 ~~0 0 7~~

enter no of frame : 4

choose Page Algorithm

1) FIFO

2) LRU

3) optimal

Page 7 → 7 - - -

Page 0 → 7 0 - -

1 → 7 0 1 -

2 → 7 0 1 2

0 → 7 0 1 2

3 → 3 0 1 2

0 → 3 0 1 2

4 → 3 4 1 2

2 → 3 4 1 2

3 → 3 4 1 2

0 → 3 4 0 2

3 → 2 4 0 2

2 → 3 4 0 1

1 → 3 4 0 1

2 → 2 4 0 1